HACK IT SAPIENS

CIPHER

# HACKTOBERFEST 26

# HACK DAY JAIPUR

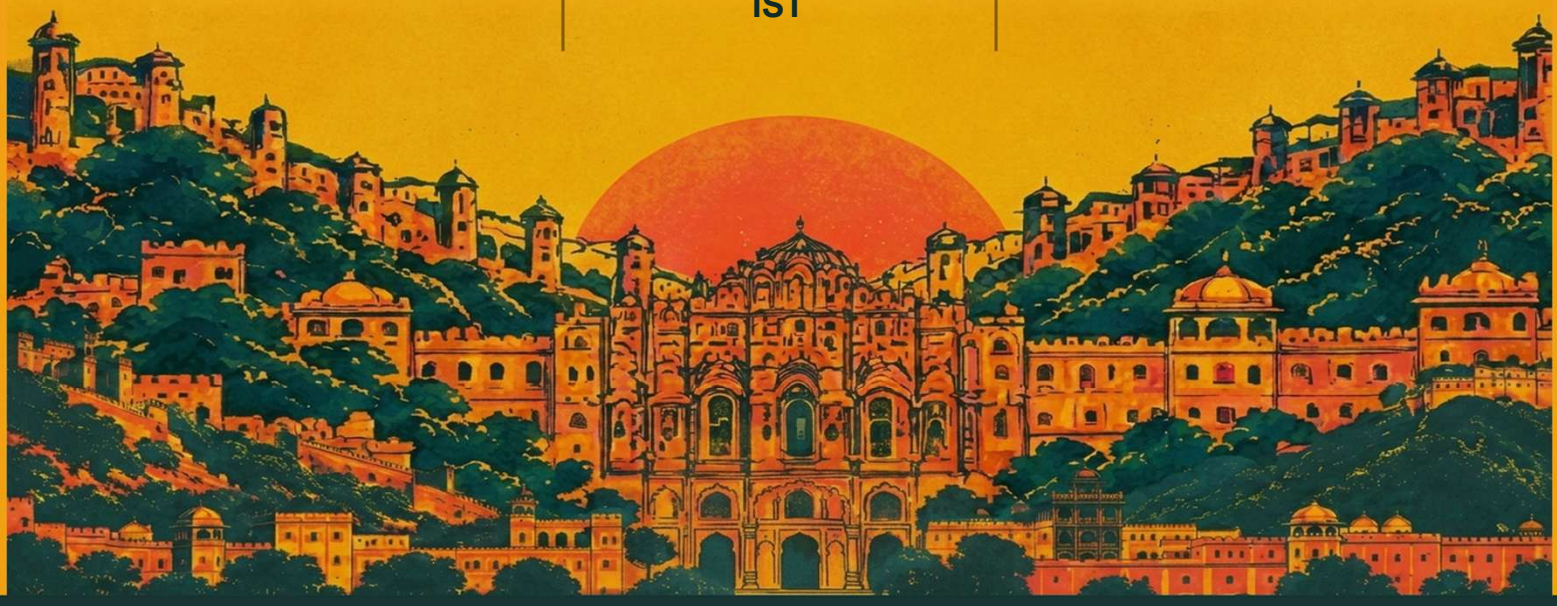
## OPEN-SOURCE AI HACKATHON

## PARTICIPATION GUIDE

25 OCT 2026

10 AM – 5 PM IST

LNMIIT, JAIPUR



Powered By MLH MAJOR LEAGUE HACKING

Presented By DigitalOcean

Gold Sponsor Graphify

AI Partner sarvam

# 1 EVENT OVERVIEW

Hack Day Jaipur is an in-person, one-day hackathon for students and early-stage builders to build with open-source AI. Explore, experiment, and ship real projects using open-weight models and modern AI tools.

### Date
25 October 2026 (Sunday)
Check-in opens at 9:00 AM IST (arrive early).

### Time
10:00 AM – 5:00 PM IST
Build starts 10:30 AM; hard cutoff at 3:30 PM.

### Venue
LNMIIT, Jaipur
In-person build sprint, mentoring, and judging.

### Format
In-person • Team or individual
(2–4 recommended; each member registers individually).

## TRACKS

### Local Inference Agents
Core inference runs on team hardware. No hosted/frontier APIs (OpenAI, Claude, Gemini) in final submission. Record model name, parameter count, hardware tested, and 3 repeat runs (tokens/sec is a tie-breaker).

### Distributed Multi-Agent Systems
Make coordination visible across multiple agents or devices: show which component does what, how state or messages move, and how the system recovers when one component fails.

### Narrow Agentic Tools
Do one job well. Show a concrete tool call/action, a useful result, and a guardrail or recovery path. Avoid presenting a general-purpose chatbot as the whole project.

### Physical AI (Edge & Embedded)
Inference runs on the physical device and works completely offline during the demo. Use an open-weight model meaningfully with real sensors, camera, or hardware controls.

### The Target: Small, Working & Testable Beats Slideware
A small, working, testable project beats a grand idea that only works in slides. Get the core flow working early, then improve it.

### Three Non-Negotiables for Every Team
1. **Public GitHub Repository:** Publish your code in a public GitHub repository with an open-source licence (MIT / Apache-2.0).
2. **Mandatory Graphify Graph:** Every team must use Graphify to generate and share a project codebase graph in Discord.
3. **Track Rules Compliance:** Follow your chosen track's specific model, offline execution, and submission rules.

**Read Before Event Day:** Download required models and dependencies in advance; do not plan to spend the build window setting up your laptop.

# 2 EVENT TIMELINE

**Know the clock. Protect build time. Check-in opens at 9:00 AM; build starts at 10:30 AM.**

| TIME | WHAT HAPPENS | YOUR JOB |
|---|---|---|
| **9:00 – 10:00** | **Check-in & Registration (Badges, seats, setup)** | Desk open till 10:30; arrive early to set up laptop. |
| **10:00 – 10:10** | **Opening & Welcome (Event, tracks, sponsors)** | Listen for venue, safety, and sponsor announcements. |
| **10:10 – 10:20** | **Rules & Track Briefing (Submission & judging)** | Note submission requirements and 3:30 PM deadline. |
| **10:20 – 10:30** | **Track Selection (Confirm team & final track)** | Agree on narrow problem and write a 1-sentence goal. |
| **10:30 – 12:00** | **Build Sprint 1 (Ideate, plan & start building)** | Set up repo (first 45m); build thinnest end-to-end flow. |
| **12:00 – 12:15** | **Lightning Talk by Saif (Short talk + Q&A)** | Time-boxed talk; keep working notes ready. |
| **12:15 – 1:30** | **Build Sprint 2 (Build with mentor support)** | Integrate local model/tools and test the main flow. |
| **1:30 – 2:00** | **Snacks & Refreshments (Food & recharge)** | At least one teammate stays available for announcements! |
| **2:00 – 3:10** | **Final Build Sprint (Complete & prepare demo)** | Handle failures, log 3 runs, run Graphify, rehearse demo. |
| **3:10 – 3:30** | **Final Packaging & Submission (Push & submit)** | Push code; finish README, Graphify & form by 3:30 PM. |
| **3:30 – 4:30** | **Judging (Review & finals)** | Be available when called; keep your demo ready. |
| **4:30 – 4:50** | **Results & Awards (Winners & recognition)** | Stay for announcements and sponsor acknowledgements. |
| **4:50 – 5:00** | **Closing & Group Photo (Final remarks)** | Follow venue instructions and clear your work area. |

### Before You Leave Home

Charge your laptop, bring its charger, sign in to GitHub, test your development environment, and download the model(s) you plan to use. Model downloads and package installation can eat your build time.

# 3 HACKATHON 101

**A quick guide for first-time builders.**

### What is a hackathon?
A short, focused event where you build a working prototype, learn new skills, meet people, and have fun.

### Who can participate?
Students, early-stage builders, and anyone interested in AI. Individual or teams of 2–4 (register individually).

### What do you build?
A working prototype or demo that solves a real problem using AI and open-source tools.

### What do you need?
A laptop, charger, pre-downloaded models, basic tools, and enthusiasm. No prior experience is required.

### How to get started? (A simple five-step plan)

**1. Choose a narrow problem:** Explore tracks, form a team, and name a real user, pain point, and outcome (e.g., 'Turn a whiteboard photo into searchable notes').

**2. Define success before coding:** Write a test you can pass at the demo: 'Given this input, return this output in under X seconds.'

**3. Pick the simplest stack you understand:** Choose one UI, one backend if needed, one model/runtime. Avoid learning 3 frameworks at once.

**4. Build the core path first:** Make one complete workflow work from input to result before adding error handling or a second feature.

**5. Freeze, test, explain:** Stop adding features before the cutoff. Run the demo 3 times, document limitations, and ensure reproducibility.

## Useful team roles (one person can cover more than one)

**Product / demo lead:** Keeps the scope small; prepares the problem statement and live demo.
**Model / agent lead:** Makes inference and tool use work; records model and runtime details.
**App / integration lead:** Builds the interface and connects components.
**Quality / release lead:** Tests the main flow (3 runs), manages GitHub, Graphify, README, and submission.

### What not to do

- **Do not** spend the first three hours debating a tech stack. Pick a working path and start.
- **Do not** build a general-purpose assistant if a narrow tool can solve the problem better.
- **Do not** leave the first successful end-to-end demo until the last hour.
- **Do not** commit API keys, passwords, .env files, private data, or secrets to GitHub.
- **Do not** claim 'offline' if your app still calls a hosted model behind the scenes — test with Wi-Fi off.
- **Do not** assume free APIs are unlimited (check quotas/terms) or disappear as a whole team during snacks.

**A Good Rule of Thumb:** By the first 45 minutes, your repository and dev environment should work. By the halfway point, have a minimal demo. Use remaining time to make it reliable.

# 4 RULES & ELIGIBILITY

## General Rules

- **Build original projects during the event:** All core implementation must occur during the hackathon.
- **Use open-source or open-weight models:** Open models must play a meaningful role — do not add a model as decoration.
- **No hosted or frontier model APIs in final submission** (e.g., OpenAI, Claude, Gemini, etc.) for local-inference and offline tracks.
- **Integrate Graphify in every project:** Generate and share your codebase graph in Graphify's official Discord.
- **Submit a public GitHub repository under an open-source licence** (MIT or Apache-2.0); include third-party and model licences separately.
- **Keep secrets out of GitHub:** Add .env to .gitignore; provide .env.example with placeholder variable names only.
- **Full technical disclosure:** Explain which model you use, where inference runs, APIs called, hardware tested, and limitations.
- **Follow track & standard requirements:** Agent skills follow agentskills.io/specification (SKILL.md); harnesses need original code or meaningful changes.

## Eligibility

- **Open to students and early-stage builders.**
- **Individual participation allowed,** but teams of 2–4 are recommended (each member registers individually).
- **Participants must follow** the code of conduct and venue rules.

## Not Allowed

- ✕ **Using hosted model APIs** as the core inference engine (in local/offline tracks).
- ✕ **Plagiarism** or copying existing projects.
- ✕ **Submitting incomplete** or non-functional projects.
- ✕ **Violating the code of conduct** or committing secrets.

## Track-specific restrictions

**Local-inference track:** Core model inference runs on the team's hardware. No hosted/frontier generative APIs such as OpenAI, Anthropic, or Gemini in the final submission. Record model name, parameter count, minimum tested hardware, and three repeat runs. Tokens/second is a tie-breaker where specified.

**Physical AI:** Inference runs on the physical device and works completely offline during the demo. Use an open-source/open-weight model meaningfully; show the device interacting with real inputs such as sensors, camera, or hardware controls.

**Other tracks / optional services:** Use the written rules for your track. External non-generative APIs may be permitted, but check connectivity, quota, terms, and whether the track permits hosted inference.

# 5 SUBMISSION GUIDE

**Submit by 3:30 PM IST on 25 October 2026. No extensions.**

## What to submit

**1. Public GitHub repository:** With source code, clear README with setup/run instructions another person can follow, open-source licence file, and zero secrets/tokens.

**2. Submission form:** Project details, problem, target user, core feature, team members, track, demo link, and model information.

**3. Working demo:** Live demo, video, or instructions to run the project.

**4. Graphify integration:** Generate and share your project graph in Graphify Discord and include/link it in your README (see Page 7).

**5. Model details:** Specify the model name, parameter count, model licence, runtime, quantization, hardware/RAM used, and external APIs (local vs. internet).

**6. Results:** Include screenshots, logs, or task-success/reliability metrics from at least three repeated runs (report tokens/second if measured).

## README.md Template (recommended)

**# Project Name**

Short description of your project. | **Track:** [Track] | **Team:** [Name + members]

**## Problem Statement & Target User**

Who has the problem and what makes it painful?

**## Approach, Architecture & Demo**

What is the one main task? [Demo link]

**## Model Details (name, size, licence, runtime, quantization, hardware)**

- Model, parameters, licence, runtime, quantization:
- Hardware (RAM / OS / CPU or GPU) & External APIs (local vs. internet):

**## Setup Instructions**

1. Prerequisites 2. Install dependencies 3. Download model 4. Start app 5. Try example input

**## Results (screenshots / metrics from 3 runs)**

Test runs, latency (tokens/sec), reliability, and known limitations.

**## Graphify Integration (link to graph) & Future Scope / Licence**

Graph: `graphify-out/graph.html` | Shared in Discord: [yes/no] | Licence: [MIT / Apache-2.0]

# 6 REQUIRED: GRAPHIFY INTEGRATION

**Every project must integrate Graphify and share the resulting graph.**

1. **Follow the Graphify quickstart guide:**
   https://docs.graphify.com/quickstart
2. **Install Graphify (CLI or IDE integration — Python 3.10+; choose one package manager):**
   uv tool install graphifyy  # alternative: pipx install graphifyy
3. **Generate a code-only graph of your codebase from the root of your repository:**
   graphify extract . --code-only
   graphify cluster-only . --no-label
   graphify export html
   Inspect graphify-out/graph.html and the generated report/JSON to confirm it matches your final repo.
4. **Share the graph in the official Graphify Discord server** using the event-specific channel or instructions.
5. **Include the graph link** in your README.md and submission form.

🎁 **Special Graphify Award:** Two outstanding graphs will be selected for personalized swag vouchers by Graphify.

## 🔗 Useful Links

- **Graphify Documentation:**
  docs.graphify.com/quickstart
- **Quickstart Guide:**
  docs.graphify.com/quickstart
- **Installation Guide:**
  docs.graphify.com/installation
- **GitHub Repository:**
  github.com/Graphify-Labs/graphify
- **Graphify Discord Server:**
  Join via event check-in link.

## 💡 Tips

- Generate the graph after your core project structure is ready.
- Use meaningful module and function names for a clearer graph.
- Include the graph link in your README and submission form.
- You can regenerate the graph after final changes.

# 7 RESOURCES FOR FIRST-TIME BUILDERS

**A curated list to help you get started quickly.**

## ★ Starter Repositories

- **Free APIs Repository:** github.com/public-apis/public-apis (check quotas/keys).
- **skills.sh (Skillverse):** www.skills.sh — reusable skills for coding agents.
- **Skills CLI — Vercel:** github.com/vercel-labs/skills (`npx skills add owner/repo`).
- **Agent Skills Collection:** agentskills.io/specification(valid SKILL.md spec).
- **DESIGN.md:** github.com/google-labs-code/design.md — UI design tokens for agents.
- **AppyAI (reference project):** github.com/aiexplorations/ai_app_template (FastAPI + Ollama).
- **Local AI Starter:** github.com/seehiong/local-ai-starter (offline WebGPU).
- **Awesome Boilerplates:** github.com/melvin0008/awesome-projects-boilerplates (Python, Node, Lua & more).

## + Graphify Resources

- **Quickstart Guide:**
  docs.graphify.com/quickstart
- **Installation Guide:**
  docs.graphify.com/installation
- **GitHub Repository:**
  github.com/Graphify-Labs/graphify
- **Discord Server:**
  Share your project graph in the official channel.

## Local AI Models (4–12 GB RAM)

- **Small LLMs:** e.g., Llama 3.2 (1B/3B), Qwen3 (0.6B/1.7B/4B), Phi, Gemma 3/4
- **Whisper (speech-to-text):** github.com/ggml-org/whisper.cpp
- **Stable Diffusion:** Local image generation
- **Embedding models:** e.g., bge-small
- **Easy local setup:** Use Ollama (docs.ollama.com) or LM Studio

## ⚙ Other Useful Resources

- **Sarvam AI Docs (Indian language AI):** docs.sarvam.ai/api-reference/introduction (permitted tracks only; not local inference).
- **Sarvam Credits:** click-here
- **Ollama Documentation:** docs.ollama.com
- **LM Studio:** Local desktop model runner
- **Hugging Face Models:** Open-weight model hub
- **Example Projects & Tutorials:** Pre-test before event day.

🌿 See Page 9 for recommended local models and setup instructions.

# 8 LOCAL MODELS & VOICE SETUP

## Choose a model your laptop can handle (4–12 GB RAM)

- **4 GB RAM (~0.5–1.3 GB download):** Qwen3 0.6B; Gemma 3 1B; Llama 3.2 1B — Short extraction, classification, routing, rewriting, simple narrow tasks.
- **8 GB RAM (~1.4–2.0 GB download):** Qwen3 1.7B; Llama 3.2 3B — Focused text tasks, small agent workflows, structured output.
- **8–12 GB RAM (~2.5–3.4 GB download):** Qwen3 4B; Gemma 3 4B — Better capability; Gemma 3 4B also supports image input in compatible runtimes.
- **12 GB, test first (~4.6 GB+):** Gemma 4 E2B (quantized/local build) — Multimodal experiments; can be tight once OS, context and apps use RAM.

**Model Pages:** ollama.com/library/qwen3 • llama3.2 • gemma3 • gemma4

### Run a local text model with Ollama

```
ollama run qwen3:1.7b
# or, on a stronger laptop:
ollama run llama3.2:3b
# or, for image + text tasks:
ollama run gemma3:4b
```

### Call the model from your app (localhost:11434)

```
curl http://localhost:11434/api/chat -d '{
  "model": "qwen3:1.7b",
  "messages": [{"role":"user",
    "content":"Return JSON label"}],
  "stream": false
}'
```

## Offline speech-to-text (Whisper & whisper.cpp)

- **Whisper tiny (39M params | ~75 MiB):** Lowest-resource experiments; may miss more words.
- **Whisper base (74M params | ~142 MiB):** Good initial choice for lower-memory laptops.
- **Whisper small (244M params | ~466 MiB):** Better accuracy, more compute; try on 8 GB+.
- **Whisper medium (769M params | ~1.5 GiB):** Try on 12 GB systems, after benchmarking.

### Other on-device considerations & Whisper 4B correction

- **Correction:** There is no standard 'Whisper 4B' model. Official sizes are tiny, base, small, medium, large, and turbo (.en is English-only; use multilingual for Indian speech).
- **Mac-specific option:** Apple Foundation Models runs on-device on supported Macs, but weights are NOT open-weight — do not use as your only model in open-weight tracks.
- **Test Before the Event:** Record model licences in README, disconnect Wi-Fi, and verify your demo works offline.

# 9 JUDGING & PROJECT DEMOS

## What judges will look for

1. **Task success / functionality:** A real input goes through the system and produces a useful result.
2. **AI is meaningful:** The open-source/open-weight model contributes materially; explain where and why it is used.
3. **Reliability:** Repeat the core workflow; explain failure handling and known limitations.
4. **Practical usefulness:** Show who benefits, when they would use it, and why this beats doing nothing or a simple non-AI script.
5. **Implementation and reproducibility:** The repo is understandable, the model and hardware are disclosed, and setup steps are credible.

## Track-specific emphasis

- **Local inference agent:** Task success rate, reliability, and practical usefulness. Show three repeated runs; tokens/second is a tie-breaker where specified.
- **Distributed / multi-agent systems:** Make coordination visible. Show which agent/device does what, how messages or state move, and what happens when one component fails.
- **Narrow agentic tool:** Do one job well. Show a tool call/action, a useful result, and a guardrail or recovery path. Avoid presenting a general-purpose chatbot as the whole project.
- **Physical AI:** Functionality 40%; offline operation 30%; practical usefulness 20%; implementation and demo 10%. Show that inference runs on the physical device without internet during the demo.

## A simple 3-minute demo plan

**0:00 – 0:30** | Problem and target user: what is broken or time-consuming today?
**0:30 – 1:45** | Live demo: input -> model/tool/device -> useful result.
**1:45 – 2:20** | Explain the AI component, model size, and why it fits the track.
**2:20 – 2:45** | Show one test result, reliability evidence, or real hardware interaction.
**2:45 – 3:00** | Mention one limitation and what you would build next.

**Your North Star:** Build the smallest useful thing, make it work, prove it works, and make it easy for someone else to understand. Bring your charger, rehearse the demo, have Graphify ready, and keep at least one teammate available during breaks.

HACKTOBERFEST 26 • HACK DAY JAIPUR x HACK IT SAPIENS

# Build the smallest useful thing, make it work, prove it works, and make it easy to understand.

25 October 2026 • LNMIIT, Jaipur • Submit by 3:30 PM IST